

Singly Linked List Operations

Source Code

```
#include <iostream>
using namespace std;

class node{
public:
    int data;
    node* next;
    node(int value=0, node* _next=nullptr){
        this->data = value;
        this->next = _next;
    }
    void display(){
        cout << "At address: " << this << endl;
        cout << "Data: " << data << endl;
        cout << "Next Address: " << next << endl;
        cout <<
        "~~~~~"
        << endl;
    }
};

class LinkedList{
public:
    node* start;
    LinkedList(node* startptr = NULL){
        this->start = startptr;
    }
    void traverse_display(){
        cout << "Current List Status:" << endl;
        node* cptr = start;
        if (start==nullptr){
            cout << "|null | null|" << endl;
        }
        else{
            while (cptr != nullptr){
                if (cptr->next == nullptr){
                    cout << "|" << cptr->data << "|" << "null" << "|";
                }
                else{
                    cout << "|" << cptr->data << "|" << cptr->next <<
                    "|" << "----->" ;
                }
                cptr = cptr->next;
            }
            cout << endl;
        }
        cout <<
```

```
"
n" << endl;
}
bool ins_beginning(int value){
    node* new_ptr = new node;
    if (new_ptr==nullptr) {
        cout << "New Node creation failed. OOM" << endl;
        return 1;
    }
    new_ptr->data = value;
    if (start == nullptr){
        start = new_ptr;
    }
    else{
        new_ptr->next = start;
        start = new_ptr;
    }
    cout << "Successfully added a new node at beginning! The new
node added is:" << endl;
    new_ptr->display();
    return 0;
}
bool ins_end(int value){
    node* new_ptr = new node(value);
    if (new_ptr==nullptr) {
        cout << "New Node creation failed. OOM" << endl;
        return 1;
    }
    if (start == nullptr){
        start = new_ptr;
    }
    else{
        node* ptr = start;
        while (ptr->next!=nullptr){
            ptr = ptr->next;
        }
        ptr->next = new_ptr;
    }
    cout << "Successfully added a new node at end! The new node
added is:" << endl;
    new_ptr->display();
    return 0;
}
bool ins_after_specific(int value, int target){
    node* new_ptr = new node;
    if (new_ptr==nullptr) {
        cout << "New Node creation failed. OOM" << endl;
        return 1;
    }
    new_ptr->data = value;
    if (start == nullptr){
        start = new_ptr;
    }
    else{
```

```

        node* ptr = start;
        while (ptr!=nullptr && ptr->data!=target){
            ptr = ptr->next;
        }
        if (ptr==nullptr) {
            cout << "Target node not found!!" << endl;
            return 1;
        }
        new_ptr->next = ptr->next;
        ptr->next = new_ptr;
    }
    cout << "Successfully added a new node after specified node: "
<< target<<" ! The new node added is:" << endl;
    new_ptr->display();
    return 0;
}

bool ins_before_specific(int value, int target){
    node* new_ptr = new node;
    if (new_ptr==nullptr) {
        cout << "New Node creation failed. OOM" << endl;
        return 1;
    }
    new_ptr->data = value;
    if (start == nullptr){
        start = new_ptr;
    }
    else{
        node* ptr = start;
        node* preptr = start;
        while (ptr!=nullptr && ptr->data!=target){
            preptr = ptr;
            ptr = ptr->next;
        }
        if (ptr==nullptr) {
            cout << "Target node not found!!" << endl;
            return 1;
        }
        new_ptr->next = ptr;
        preptr->next = new_ptr;
    }
    cout << "Successfully added a new node before specified node: "
<< target<<" ! The new node added is:" << endl;
    new_ptr->display();
    return 0;
}

bool ins_at_n(int value, int n){
    node* new_ptr = new node;
    if (new_ptr==nullptr) {
        cout << "New Node creation failed. OOM" << endl;
        return 1;
    }
    new_ptr->data = value;
    if (start == nullptr){
        start = new_ptr;
    }

```

```
    }
    else{
        if (n==1){
            new_ptr->next = start;
            start = new_ptr;
        }
        else{
            node* ptr = start;
            node* preptr = start;
            for (int i=0;i<n-1;i++){
                preptr = ptr;
                ptr = ptr->next;
            }
            if (ptr==nullptr) {
                cout << "Target node not found!!" << endl;
                return 1;
            }
            new_ptr->next = ptr;
            preptr->next = new_ptr;
        }
    }
    cout << "Successfully added a new node at "<< n << "th
position! The new node added is:" << endl;
    new_ptr->display();
    return 0;
}

bool del_beginning(){
    if (start==nullptr){
        cout << "The list is empty: Underflow!!" << endl;
        return 1;
    }
    else{
        node* temp = start;
        start = start->next;
        cout << "Successfully deleted the node at beginning. The
deleted node was:" << endl;
        temp->display();
        delete temp;
    }
    return 0;
}

bool del_end(){
    if (start==nullptr){
        cout << "The list is empty: Underflow!!" << endl;
        return 1;
    }
    else{
        node* temp = start;
        node* pretemp = start;
        while (temp->next != nullptr){
            pretemp = temp;
            temp = temp->next;
        }
        pretemp->next = nullptr;
    }
}
```

```

        cout << "Successfully deleted the node at end. The deleted
node was:" << endl;
        temp->display();
        delete temp;
    }
    return 0;
}

bool del_after_specific(int target){
    if (start==nullptr){
        cout << "The list is empty: Underflow!!" << endl;
        return 1;
    }
    else{
        node* pretemp = start;
        while (pretemp!=nullptr && pretemp->data != target){
            pretemp = pretemp->next;
        }
        //cout << "loop exited!" << endl;
        if (pretemp==nullptr){
            cout << "Node "<< target <<" not found!!" << endl;
            return 1;
        }
        if (pretemp->next != nullptr){
            node* temp = pretemp->next;
            pretemp->next = temp->next;
            cout << "Successfully deleted after specified node: "<<
target <<" . The deleted node was:" << endl;
            temp->display();
            delete temp;
        }
        else{
            cout << "No node after "<< target <<"! What shall I
delete?" << endl;
            return 1;
        }
    }
    return 0;
}

bool del_at_n(int n){
    if (start==nullptr){
        cout << "The list is empty: Underflow!!" << endl;
        return 1;
    }
    else{
        if (n==1){
            node* temp = start;
            start = start->next;
            cout << "Successfully deleted the node at "<< n << "th
position! The deleted node was:" << endl;
            temp->display();
            delete temp;
        }
        else{

```

```

        node* temp = start;
        node* pretemp = start;
        for (int i=0;i<n-1;i++){
            pretemp = temp;
            temp = temp->next;
        }
        if (temp==nullptr) {
            cout << "No node at index " << n <<" found!!" <<
endl;

            return 1;
        }
        pretemp->next = temp->next;
        cout << "Successfully deleted the node at "<< n << "th
position! The deleted node was:" << endl;
        temp->display();
        delete temp;
    }
    return 0;
}

};

int main(){
    LinkedList myLL;
    myLL.traverse_display();

    cout << "Insertion at Beginning: " << endl;
    myLL.ins_beginning(2);
    myLL.traverse_display();

    cout << "Insertion at End: " << endl;
    myLL.ins_end(3);
    myLL.traverse_display();

    cout << "Insertion before element 3: " << endl;
    myLL.ins_before_specific(4, 3);
    myLL.traverse_display();

    cout << "Insertion after element 3: " << endl;
    myLL.ins_after_specific(5, 3);
    myLL.traverse_display();

    cout << "Insertion at index 3: " << endl;
    myLL.ins_at_n(6, 3);
    myLL.traverse_display();

    cout << "Deletion at Beginning: " << endl;
    myLL.del_beginning();
    myLL.traverse_display();

    cout << "Deletion at End: " << endl;
    myLL.del_end();
    myLL.traverse_display();

```

```

    cout << "Deletion after 4: " << endl;
    myLL.del_after_specific(4);
    myLL.traverse_display();

    cout << "Deletion after 20: " << endl;
    myLL.del_after_specific(20);
    myLL.traverse_display();

    cout << "Deletion after 3: " << endl;
    myLL.del_after_specific(3);
    myLL.traverse_display();

    cout << "Deletion at index 1: " << endl;
    myLL.del_at_n(1);
    myLL.traverse_display();

    cout << "Deletion at Beginning: " << endl;
    myLL.del_beginning();
    myLL.traverse_display();

    cout << "Deletion at End: " << endl;
    myLL.del_end();
    myLL.traverse_display();

    return 0;
}

```

Output

```

pawanadhikari@Pawans-MacBook-Air 04_SLL % ./01_SinglyLinkedList
Current List Status:
|null | null|

```

```

Insertion at Beginning:
Successfully added a new node at beginning! The new node added is:
At address: 0x10338df70
Data: 2
Next Address: 0

```

```

~~~~~
Current List Status:
|2|null|

```

```

Insertion at End:
Successfully added a new node at end! The new node added is:
At address: 0x10338df80
Data: 3
Next Address: 0

```

```

~~~~~
Current List Status:

```

```
|2|0x10338df80|----->|3|null|
```

Insertion before element 3:

Successfully added a new node before specified node: 3 ! The new node added is:

At address: 0x10338dca0

Data: 4

Next Address: 0x10338df80

~~~~~  
Current List Status:

```
|2|0x10338dca0|----->|4|0x10338df80|----->|3|null|
```

---

Insertion after element 3:

Successfully added a new node after specified node: 3 ! The new node added is:

At address: 0x10338dcb0

Data: 5

Next Address: 0

~~~~~  
Current List Status:

```
|2|0x10338dca0|----->|4|0x10338df80|----->|3|0x10338dcb0|-----  
>|5|null|
```

Insertion at index 3:

Successfully added a new node at 3th position! The new node added is:

At address: 0x10338dcc0

Data: 6

Next Address: 0x10338df80

~~~~~  
Current List Status:

```
|2|0x10338dca0|----->|4|0x10338dcc0|----->|6|0x10338df80|-----  
>|3|0x10338dcb0|----->|5|null|
```

---

Deletion at Beginning:

Successfully deleted the node at beginning. The deleted node was:

At address: 0x10338df70

Data: 2

Next Address: 0x10338dca0

~~~~~  
Current List Status:

```
|4|0x10338dcc0|----->|6|0x10338df80|----->|3|0x10338dcb0|-----  
>|5|null|
```

Deletion at End:

Successfully deleted the node at end. The deleted node was:

At address: 0x10338dcb0

Data: 5

Next Address: 0

~~~~~

Current List Status:

|4|0x10338dcc0|----->|6|0x10338df80|----->|3|null|

---

Deletion after 4:

Successfully deleted after specified node: 4 . The deleted node was:

At address: 0x10338dcc0

Data: 6

Next Address: 0x10338df80

~~~~~  
Current List Status:

|4|0x10338df80|----->|3|null|

Deletion after 20:

Node 20 not found!!

Current List Status:

|4|0x10338df80|----->|3|null|

Deletion after 3:

No node after 3! What shall I delete?

Current List Status:

|4|0x10338df80|----->|3|null|

Deletion at index 1:

Successfully deleted the node at 1th position! The deleted node was:

At address: 0x10338dca0

Data: 4

Next Address: 0x10338df80

~~~~~  
Current List Status:

|3|null|

---

Deletion at Beginning:

Successfully deleted the node at beginning. The deleted node was:

At address: 0x10338df80

Data: 3

Next Address: 0

~~~~~  
Current List Status:

|null | null|

Deletion at End:

The list is empty: Underflow!!

Current List Status:

|null | null|
